



Workshop 3: Naming

Distributed Systems — Practical Report

Authors: Dario Pomasqui & Adriana Sofia Espinoza Chicaiza

School of Mathematical and Computational Sciences, Yachay Tech University

Professor: Francisco Hidrobo, Ph.D. | **Date:** September 03, 2026

GitHub Repository:

github.com/adrianasofiaespinoza/Sistemas-Distribuidos/tree/main/Workshop3

1 Theoretical Framework

In distributed systems, accessing resources and communication endpoints fundamentally requires resolving an identification token to an operational access point [1, 2]. Literature distinguishes three core concepts:

- **Name:** A human-friendly or symbolic label for an entity (e.g., DNS hostname `student-a.lan`).
- **Identifier:** A location-independent, unique bit string (e.g., UUIDv5) that refers to at most one entity, is never reused, and never changes even when the entity relocates.
- **Address:** A location-dependent network contact point (e.g., IPv4 tuple (192.168.1.14, 5000) or MAC address) used by routing protocols to deliver packets.

Naming Paradigms Evaluated in Workshop 3

- **Flat Naming:** Identifiers have no internal structure or topological clues. Resolution relies on: (1) *Broadcasting* (LAN-only flooding), (2) *Forwarding Pointers* (movement chains with shortcut optimizations), and (3) *Distributed Hash Tables (DHTs)* like Chord ($O(\log N)$ routing).
- **Structured / Hierarchical Naming:** Names are organized into hierarchical domain trees (e.g., Hierarchical Location Services [HLS]) or directed naming graphs (e.g., POSIX file systems with hard and soft links).

2 Exercise 1 — Identifiers vs. Addresses

Objective: Decouple entity identity from network location and verify that an entity identifier remains invariant under address reassignment.

Implementation and Evidence: In `identifiers.py`, a deterministic UUIDv5 is generated from "student-a" in the DNS namespace, while the IP address is discovered dynamically. The entity records its address at port 5000 and is subsequently reassigned to port 6000:

```

Exercise 1 - Real execution of identifiers.py

PS C:\Users\User\Desktop\Sistemas-Distribuidos\Sistemas-Distribuidos\Workshop3> python identifiers.py
Entity ID : fc1ce712-f56d-5831-be86-2b82ea445f5a
Hostname  : DESKTOP-MLJE0GD
Address   : 192.168.1.14
Initial entity state: {'id': 'fc1ce712-f56d-5831-be86-2b82ea445f5a', 'address': ('192.168.1.14', 5000)}
Updated entity state: {'id': 'fc1ce712-f56d-5831-be86-2b82ea445f5a', 'address': ('192.168.1.14', 6000)}
  
```

Figure 1: Terminal output of `identifiers.py`: the UUID remains constant while the port address is modified.

Why UUID Alone is Insufficient for Communication

When Student A provides only the UUID (fc1ce712-f56d-5831-be86-2b82ea445f5a), Student B cannot initiate direct communication because:

1. **Zero Routing Semantics:** Transport and network layer protocols require topological endpoints (IP address, transport port, MAC address). A UUID contains no subnet or gateway routing data.
2. **Separation of Concerns:** An identifier specifies *which* entity is targeted, while an address specifies *where* it is currently hosted.
3. **Mobility and Rebinding:** When the port changes from 5000 to 6000, communication without an active lookup service would fail. An explicit resolution mechanism ($\text{UUID} \mapsto (\text{IP}, \text{Port})$) is mandatory.

3 Exercise 2 — Locating an Entity by Broadcast

Objective: Study flat name resolution through broadcast at the Link Layer (ARP) and Application Layer (UDP sockets).

3.1 Link-Layer Resolution (ARP / Neighbor Cache)

Inspecting the operating system's ARP table (`arp -a`) reveals the dynamic binding between logical IPv4 addresses and physical Layer-2 MAC addresses. Pinging the local gateway triggers ARP requests and populates the cache:

```

Exercise 2 - Real ARP and ping evidence

PS C:\Users\User\Desktop\Sistemas-Distribuidos\Sistemas-Distribuidos\Workshop3> ipconfig | findstr /i IPv4
IPv4 Address. . . . . : 192.168.1.14
IPv4 Address. . . . . : 172.18.16.1

PS C:\Users\User\Desktop\Sistemas-Distribuidos\Sistemas-Distribuidos\Workshop3> ipconfig | findstr /i Gateway
Default Gateway . . . . . : 192.168.1.1
Default Gateway . . . . . :

PS C:\Users\User\Desktop\Sistemas-Distribuidos\Sistemas-Distribuidos\Workshop3> arp -a

Interface: 192.168.1.14 --- 0xf
Internet Address      Physical Address      Type
192.168.1.1           84-d3-43-8f-72-3c    dynamic
192.168.1.8           8c-7a-3d-ea-94-b4    dynamic
192.168.1.12          a2-f7-8f-2b-5c-43    dynamic
192.168.1.16          66-5d-9e-d2-08-bb    dynamic
192.168.1.255         ff-ff-ff-ff-ff-ff    static
224.0.0.2             01-00-5e-00-00-02    static
224.0.0.22            01-00-5e-00-00-16    static
224.0.0.251           01-00-5e-00-00-fb    static
224.0.0.252           01-00-5e-00-00-fc    static
239.255.255.250       01-00-5e-7f-ff-fa    static
255.255.255.255       ff-ff-ff-ff-ff-ff    static

Interface: 172.18.16.1 --- 0x30
Internet Address      Physical Address      Type
172.18.24.189         00-15-5d-84-50-42    dynamic
172.18.31.255         ff-ff-ff-ff-ff-ff    static
224.0.0.22            01-00-5e-00-00-16    static
224.0.0.251           01-00-5e-00-00-fb    static
239.255.255.250       01-00-5e-7f-ff-fa    static

PS C:\Users\User\Desktop\Sistemas-Distribuidos\Sistemas-Distribuidos\Workshop3> ping 192.168.1.1 -n 2

Pinging 192.168.1.1 with 32 bytes of data:
Reply from 192.168.1.1: bytes=32 time=2ms TTL=64
Reply from 192.168.1.1: bytes=32 time=2ms TTL=64

Ping statistics for 192.168.1.1:
    Packets: Sent = 2, Received = 2, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 2ms, Maximum = 2ms, Average = 2ms

PS C:\Users\User\Desktop\Sistemas-Distribuidos\Sistemas-Distribuidos\Workshop3> arp -a

Interface: 192.168.1.14 --- 0xf
Internet Address      Physical Address      Type
192.168.1.1           84-d3-43-8f-72-3c    dynamic
192.168.1.8           8c-7a-3d-ea-94-b4    dynamic
192.168.1.12          a2-f7-8f-2b-5c-43    dynamic
192.168.1.16          66-5d-9e-d2-08-bb    dynamic
192.168.1.255         ff-ff-ff-ff-ff-ff    static
224.0.0.2             01-00-5e-00-00-02    static
224.0.0.22            01-00-5e-00-00-16    static
224.0.0.251           01-00-5e-00-00-fb    static
224.0.0.252           01-00-5e-00-00-fc    static
239.255.255.250       01-00-5e-7f-ff-fa    static
255.255.255.255       ff-ff-ff-ff-ff-ff    static

Interface: 172.18.16.1 --- 0x30
Internet Address      Physical Address      Type
172.18.24.189         00-15-5d-84-50-42    dynamic
172.18.31.255         ff-ff-ff-ff-ff-ff    static
224.0.0.22            01-00-5e-00-00-16    static
224.0.0.251           01-00-5e-00-00-fb    static
239.255.255.250       01-00-5e-7f-ff-fa    static

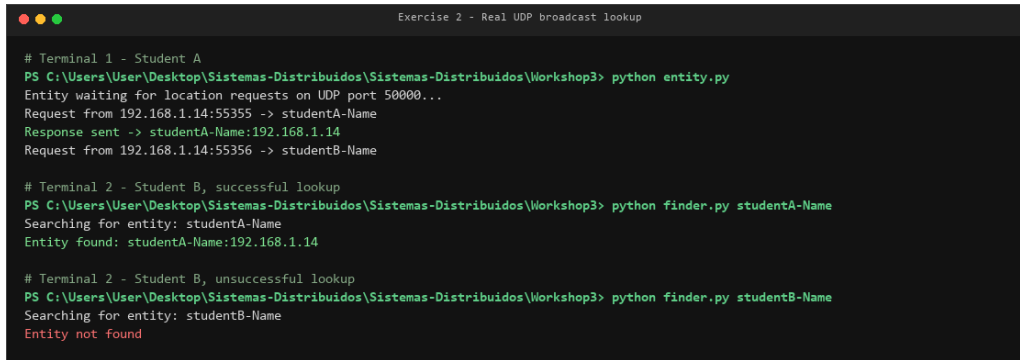
```

Figure 2: Link-layer evidence: IPv4 configuration, ARP neighbor table, and ICMP ping verification.

3.2 Application-Level UDP Broadcast

At the application layer, `entity.py` (Student A) listens on UDP port 50000, while `finder.py` (Student B) broadcasts to 255.255.255.255:50000.

- **Successful Discovery:** Querying "studentA-Name" returns "studentA-Name:192.168.1.14".
- **Unsuccessful Discovery:** Querying non-existent "studentB-Name" causes a 3-second socket timeout resulting in "Entity not found".



```

Exercise 2 - Real UDP broadcast lookup

# Terminal 1 - Student A
PS C:\Users\User\Desktop\Sistemas-Distribuidos\Sistemas-Distribuidos\Workshop3> python entity.py
Entity waiting for location requests on UDP port 50000...
Request from 192.168.1.14:55355 -> studentA-Name
Response sent -> studentA-Name:192.168.1.14
Request from 192.168.1.14:55356 -> studentB-Name

# Terminal 2 - Student B, successful lookup
PS C:\Users\User\Desktop\Sistemas-Distribuidos\Sistemas-Distribuidos\Workshop3> python finder.py studentA-Name
Searching for entity: studentA-Name
Entity found: studentA-Name:192.168.1.14

# Terminal 2 - Student B, unsuccessful lookup
PS C:\Users\User\Desktop\Sistemas-Distribuidos\Sistemas-Distribuidos\Workshop3> python finder.py studentB-Name
Searching for entity: studentB-Name
Entity not found

```

Figure 3: Real terminal capture of UDP broadcast resolution: Student A listener, Student B success, and Student B timeout.

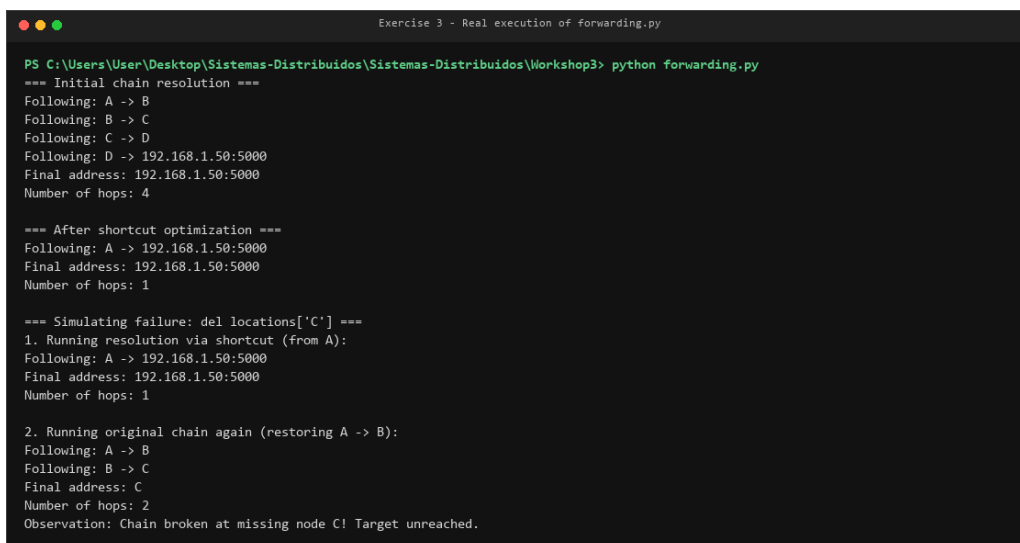
Scalability Limitations: Broadcasting does not scale in distributed systems because: (1) *CPU Interrupt Storms*: Every node on the LAN must interrupt its processor to inspect every broadcast frame ($O(N)$ network overhead); (2) *Router Boundaries*: Routers deliberately drop link-local broadcast packets, confining resolution strictly to a single local subnet.

4 Exercise 3 — Forwarding Pointers

Objective: Trace forwarding pointer chains, implement the shortcut optimization, and simulate intermediate link failure.

Implementation and Evidence: In `forwarding.py`, an entity moves through locations $A \rightarrow B \rightarrow C \rightarrow D \rightarrow \text{Address}$.

- **Unoptimized Chain:** Traversing all intermediate pointers requires **4 hops**.
- **Shortcut Optimization:** Updating A 's pointer directly to the final address (`locations["A"] = address`) compresses resolution to **1 hop** (a 75% reduction in lookup latency).
- **Failure Simulation:** Deleting intermediate node C (`del locations["C"]`) severs the original pointer chain ($A \rightarrow B \rightarrow C \rightarrow \text{Halt}$), preventing callers relying on the legacy path from reaching the target entity.



```

Exercise 3 - Real execution of forwarding.py

PS C:\Users\User\Desktop\Sistemas-Distribuidos\Sistemas-Distribuidos\Workshop3> python forwarding.py
=== Initial chain resolution ===
Following: A -> B
Following: B -> C
Following: C -> D
Following: D -> 192.168.1.50:5000
Final address: 192.168.1.50:5000
Number of hops: 4

=== After shortcut optimization ===
Following: A -> 192.168.1.50:5000
Final address: 192.168.1.50:5000
Number of hops: 1

=== Simulating failure: del locations['C'] ===
1. Running resolution via shortcut (from A):
Following: A -> 192.168.1.50:5000
Final address: 192.168.1.50:5000
Number of hops: 1

2. Running original chain again (restoring A -> B):
Following: A -> B
Following: B -> C
Final address: C
Number of hops: 2
Observation: Chain broken at missing node C! Target unreachable.

```

Figure 4: Terminal output of `forwarding.py`: 4-hop resolution, 1-hop shortcut, and chain failure simulation.

Table 1: Forwarding pointer resolution metrics

Test Configuration	Traversal Path	Hops	Resolution Status
Unoptimized Chain	$A \rightarrow B \rightarrow C \rightarrow D \rightarrow 192.168.1.50 : 5000$	4	Successful
Shortcut Optimization	$A \rightarrow 192.168.1.50 : 5000$	1	Successful (75% latency reduction)
Intermediate Failure (del C)	$A \rightarrow B \rightarrow C \rightarrow \text{HALT}$	2	Broken Chain (Failure)

Discussion: While forwarding pointers provide $O(1)$ update cost during migration, long chains accumulate network round-trip delays ($O(k)$ RTT) and suffer high vulnerability: a single crashed intermediary breaks client access permanently unless shortcuts and garbage collection are maintained.

5 Exercise 4 — Implementing a Simplified Chord Ring

Objective: Implement a Distributed Hash Table (DHT) based on Chord consistent hashing with $m = 5$ ($2^5 = 32$ positions), compute finger tables, and route lookups in $O(\log N)$ hops [3].

5.1 Mathematical Model and Ring Setup

- **Identifier Circle:** Positions $0 \dots 31 \pmod{32}$. Active nodes: $\mathcal{N} = \{1, 4, 9, 11, 14, 18, 20, 21, 28\}$.
- **Successor Rule:** Key k is stored at the smallest node $\geq k \pmod{32}$: $\text{succ}(k) = \min\{n \in \mathcal{N} \mid n \geq k\}$.
- **Finger Table:** For node p , entry i ($1 \leq i \leq 5$) points to $FT_p[i] = \text{succ}((p + 2^{i-1}) \pmod{32})$.

5.2 Successor Mapping and Finger Tables

Table 2: Part A: Key-to-Node Mapping

Key	Node	Justification
3	4	Smallest node ≥ 3
8	9	Smallest node ≥ 8
12	14	Nodes 9, 11 $< 12 \implies 14$
19	20	Smallest node ≥ 19
26	28	Smallest node ≥ 26
30	1	Wraps around modulo 32

Table 3: Part B: Finger Tables (FT_p)

Node	+1	+2	+4	+8	+16
1	4	4	9	9	18
4	9	9	9	14	20
9	11	11	14	18	28
11	14	14	18	20	28
14	18	18	18	28	1
18	20	20	28	28	4
20	21	28	28	28	4
21	28	28	28	1	9
28	1	1	1	4	14

5.3 Routing Execution and Ring Topology

- **Lookup Key 26 from Node 1:** $1 \xrightarrow{FT_1[5]} 18 \xrightarrow{FT_{18}[2]} 20 \xrightarrow{FT_{20}[1]} 21 \xrightarrow{\text{succ}} 28$ (4 hops).
- **Lookup Key 12 from Node 28:** $28 \xrightarrow{FT_{28}[4]} 4 \xrightarrow{FT_4[3]} 9 \xrightarrow{FT_9[1]} 11 \xrightarrow{\text{succ}} 14$ (4 hops).

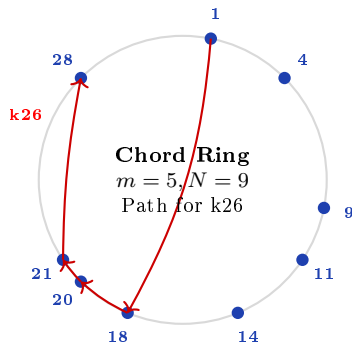


Figure 5: Chord logical ring topology.

```

Exercise 4 - Real execution of chord.py

PS C:\Users\User\Desktop\Sistemas-Distribuidos\Sistemas-Distribuidos\Workshop3> python chord.py

=== Part A - Chord definition ===
Key: 3 -> node: 4
Key: 8 -> node: 9
Key: 12 -> node: 14
Key: 19 -> node: 20
Key: 26 -> node: 28
Key: 30 -> node: 1

=== Part B - finger tables ===
Node 1 Finger Table:
1 start successor
1 2 4
2 3 4
3 5 9
4 9 9
5 17 18

Finger tables for all nodes:
Node 1: FT = [4, 4, 9, 9, 18]
Node 4: FT = [9, 9, 9, 14, 20]
Node 9: FT = [11, 11, 14, 18, 28]
Node 11: FT = [14, 14, 15, 20, 28]
Node 14: FT = [18, 18, 18, 28, 1]
Node 18: FT = [20, 20, 28, 28, 4]
Node 20: FT = [21, 28, 28, 28, 4]
Node 21: FT = [28, 28, 28, 1, 9]
Node 28: FT = [1, 1, 1, 4, 14]

=== Part C - Lookup ===
Lookup key 26 starting at 1
1
-> 18
-> 20
-> 21
-> 28
Total hops = 4

Lookup key 12 starting at 28
28
-> 4
-> 9
-> 11
-> 14
Total hops = 4

```

Figure 6: Terminal execution of chord.py.

6 Exercise 5 — Hierarchical Location Service (HLS)

Objective: Model a worldwide network hierarchy using administrative domains, downward pointers, and locality-aware search.

6.1 Architecture and Lookup Principles

The namespace is partitioned into a tree: $ROOT \rightarrow \{AMERICA, EUROPE\}$, $AMERICA \rightarrow \{ECUADOR, USA\}$, $ECUADOR \rightarrow \{IBARRA, QUITO\}$. Leaves store entity addresses; intermediate directory nodes maintain **downward pointers** to child subtrees containing entities.

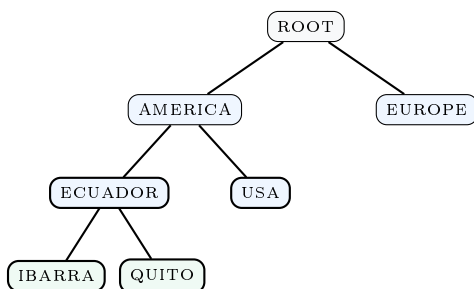


Figure 7: HLS domain tree hierarchy.

```

Exercise 5 - Real execution of hls.py

PS C:\Users\User\Desktop\Sistemas-Distribuidos\Sistemas-Distribuidos\Workshop3> python hls.py

=== Initial Lookups ===
Lookup server01 from IBARRA:
IBARRA
-> server01
Address: 10.0.1.20

Lookup server02 from IBARRA:
IBARRA
-> ECUADOR
-> QUITO
-> server02
Address: 10.0.2.30

Lookup server99 from IBARRA:
IBARRA
-> ECUADOR
-> AMERICA
-> ROOT
Entity not found
Address: None

=== Moving server01 from IBARRA to QUITO ===
server01 moved to QUITO (new address: 10.0.2.25)

Lookup server01 from IBARRA after move:
IBARRA
-> ECUADOR
-> QUITO
-> server01
Address: 10.0.2.25

```

Figure 8: Terminal execution of hls.py.

6.2 Evaluation of Locality and Entity Relocation

- Exploitation of Locality:** Resolving `server02` from `IBARRA` moves upward only to `ECUADOR` (the Lowest Common Ancestor, LCA), which contains a downward pointer directly to `QUITO`. It **never** contacts `AMERICA` or `ROOT`, completely shielding global nodes from regional query traffic.
- Search Boundary for Absent Entities:** Querying non-existent `server99` ascends all the way to `ROOT` before terminating with "Entity not found".
- Localized Migration Cost:** Relocating `server01` from `IBARRA` to `QUITO` (address 10.0.2.25) only modifies records within `ECUADOR` and the leaves. Pointers at `AMERICA` and `ROOT` remain untouched,

proving minimal update signaling overhead.

7 Exercise 6 — Name Spaces, Inodes, Hard and Soft Links

Objective: Analyze graph-theoretic namespace resolution by comparing hard links (direct inode references) with soft links (indirect symbolic path pointers) under file rename and deletion.

```

PS C:\Users\User\Desktop\Sistemas-Distribuidos\Sistemas-Distribuidos\Workshop3> wsl -d docker-desktop -- sh
/mnt/host/c/Users/User/Desktop/Sistemas-Distribuidos/Sistemas-Distribuidos/Workshop3/run_links_evidence.sh
/tmp/naming_lab_1788482375_25296
$ find .
.
./data
./data/original.txt

$ ln data/original.txt hardlink.txt
$ ln -s data/original.txt softlink.txt

$ ls -li hardlink.txt softlink.txt data/original.txt
 858 -rw-r--r--  2 root   root    20 Sep  4 00:39 data/original.txt
 858 -rw-r--r--  2 root   root    20 Sep  4 00:39 hardlink.txt
 859 lrwxrwxrwx  1 root   root    17 Sep  4 00:39 softlink.txt -> data/original.txt

$ cat hardlink.txt
Distributed Systems
$ cat softlink.txt
Distributed Systems

$ mv data/original.txt data/renamed.txt
$ cat hardlink.txt
Distributed Systems
$ cat softlink.txt
cat: can't open 'softlink.txt': No such file or directory

$ rm data/renamed.txt
$ cat hardlink.txt
Distributed Systems
$ cat softlink.txt
cat: can't open 'softlink.txt': No such file or directory

```

Figure 9: POSIX shell execution verifying inode numbers, link counts, and behavior under file renaming and deletion.

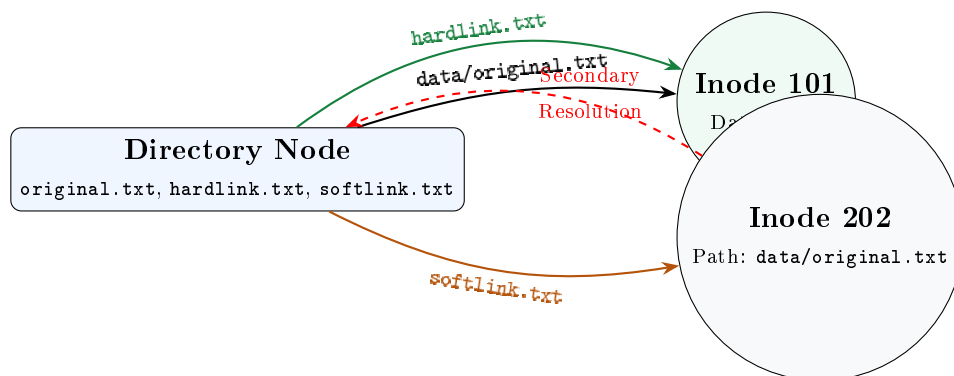


Figure 10: Naming graph illustrating direct inode binding (hard link) vs. path indirection (soft link).

Theoretical Explanation of Results

- **Hard Links Participate Directly in the Naming Graph:** `data/original.txt` and `hardlink.txt` share the identical inode number (7881299348562163) with link count 2. Renaming or deleting the original file merely unlinks that specific directory entry and decrements the inode reference count to 1; the data blocks remain fully accessible via `hardlink.txt`.
- **Soft Links Introduce Fragile Indirection:** The symbolic link is a separate inode (9288674232115449) whose payload is a text path string. Renaming `original.txt` to `renamed.txt` leaves the soft link pointing to a non-existent path string, producing an unresolvable dangling reference (ENOENT: No such file or directory).

8 Comparative Synthesis of Naming Schemes

Table 4: Dimensional comparison of distributed naming and location mechanisms

Mechanism	Lookup tency	La-	Update Cost	Scalability	Failure Vulnerability
Broadcast (ARP/UDP)	$O(1)$ (LAN)	RTT	$O(1)$ (no tables)	Low (bounded by local subnet)	Resilient to crashes; vulnerable to packet storms
Forwarding Pointers	$O(k)$ hops		$O(1)$ at prior node	Low (chains lengthen with mobility)	High (single intermediate failure severs chain)
DHT / Chord Ring	$O(\log N)$ hops		$O(\log^2 N)$ updates	High (scales to millions of nodes)	Robust through finger stabilization and successor lists
HLS (Hierarchical)	$O(\text{height})$ (to LCA)	(to LCA)	Localized to LCA	Very High (exploits geographic locality)	High-level failures impede wide-area lookups
FS Hard Links	$O(1)$ direct inode		$O(1)$ refcount	Single file system partition	High durability (data preserved while refcount > 0)
FS Soft Links	$O(\text{path length})$		$O(1)$ string write	Cross-device / distributed mounts	Fragile (renaming target yields dangling reference)

9 Conclusions

1. **Identity vs. Location:** Identifiers provide permanent, location-independent entity keys, whereas addresses provide topological routing endpoints. Effective distributed systems must cleanly decouple identity from physical access points.
2. **Flat vs. Hierarchical Trade-offs:** While broadcast provides zero-configuration discovery on LANs, it cannot cross network boundaries. Structured approaches like Chord achieve logarithmic $O(\log N)$ scalability through consistent hashing.
3. **Locality Exploitation:** Hierarchical architectures like HLS confine queries and update signaling to the lowest common ancestor (LCA), protecting higher-level infrastructure from regional bottlenecks.
4. **Graph-Theoretic Naming:** Direct inode references (hard links) provide crash resilience and data retention, whereas indirect path resolution (soft links) offers cross-filesystem flexibility at the expense of dangling pointers.

References

- [1] A. S. Tanenbaum and M. van Steen, *Distributed Systems: Principles and Paradigms*, 3rd ed., 2017.
- [2] G. Coulouris, J. Dollimore, T. Kindberg, and G. Blair, *Distributed Systems: Concepts and Design*, 5th ed., 2011.
- [3] I. Stoica et al., “Chord: A scalable peer-to-peer lookup service for internet applications,” *ACM SIGCOMM*, 2001.
- [4] F. Hidrobo, “Workshop 3: Naming — Distributed Systems,” Yachay Tech University, Semester II, 2026.